

Computer Organization & Assembly Language

Lecture 3

Programming Model x8086

- How software interacts with the hardware of x86-based systems
- How data is organized
- How instructions are executed
- How memory is accessed

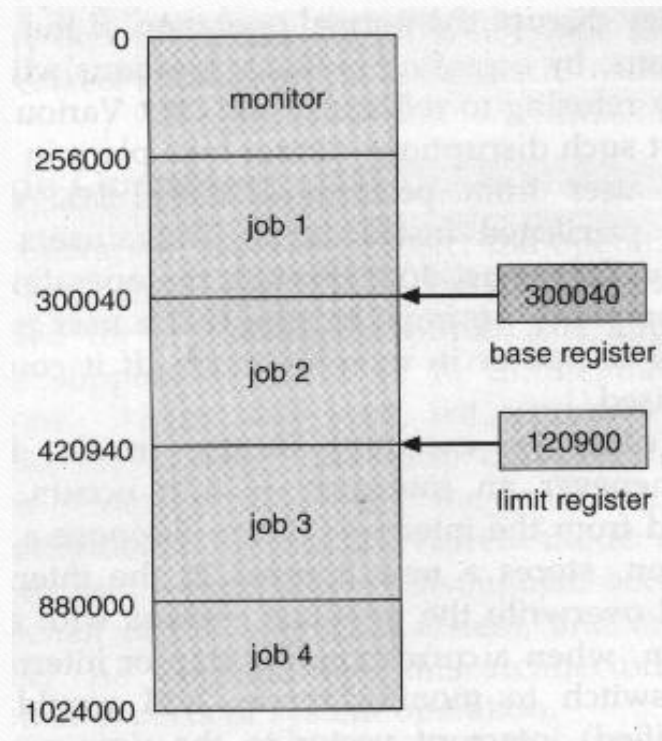
Architecture

- The x86 architecture is a family of instruction set architectures (ISAs) based on the Intel 8086 microprocessor and its descendants.
- It supports both 16-bit, 32-bit (x86), and 64-bit (x86-64) computing environments.
- Next – Registers (already discussed)

Memory

- The x86 architecture uses a segmented memory model, allowing programs to use larger address spaces by dividing memory into segments.
- Memory is divided into segments, with each segment defined by a **base address** and a **limit**. This enables different data types and instructions to be stored in separate memory areas.

Memory



Instruction Set

- **Data Movement:** Move data between registers, memory, and I/O ports.
- **Arithmetic Operations:** Perform calculations like addition, subtraction, multiplication, and division.
- **Logical Operations:** Perform bitwise operations (AND, OR, NOT, XOR).
- **Control Flow:** Manage the flow of the program using jumps, calls, and loops.
- **String Operations:** Special instructions for handling string data.

Assembler Directives

- Pseudo-ops
- Instructions in assembly language that are not converted into machine code
- Instructions that tell the assembler how to process the source code, define data segments, manage memory, and control the program's structure.
- They help organize code, define data structures, and manage memory
- Emu8086 = assembler + emulator

Data Definition Directives

- **DB (Define Byte):** Allocates space for a byte (8 bits) or initializes it with a value.
 - Value DB 10 (value is name of variable initialize to 10)
- **DW (Define Word):** Allocates space for a word (usually 16 bits) or initializes it with a value.
 - Num DW 1234
- **DD (Define Doubleword):** Allocates space for a doubleword (usually 32 bits) or initializes it with a value.
 - largeNum DD 100000
- **DQ (Define Quadword):** Allocates space for a quadword (usually 64 bits) or initializes it with a value.
 - bigNum DQ 123456789012345
- **DT (Define Ten Bytes):** Allocates space for ten bytes.
 - myArray DT 1, 2, 3, 4, 5

Section Directives

- **SECTION or SEGMENT:** Begins a new section in the program (like .text for code or .data for initialized data).
- SECTION .text
- ; code goes here
- **END:** Marks the end of the assembly file. Indicates the end of the program.

Code Control Directives

- **LABEL:** Marks a location in the code for use as a reference.
 - Labels can be any valid identifier and are typically used to indicate where certain operations or blocks of code begin.
 - start: MOV AX, 1

.model

```
assembly
```

```
.model small
```

-
- Specifies the memory model used by the program, which affects how the program accesses memory.
 - **Small:** Single segment for code and data. Suitable for small programs. Code and data are located in a single 64 KB segment. Well-suited for small programs that fit within the 64 KB limit.
 - **Medium:** One segment for code and one segment for data, allowing more code than the small model. Code and data have their own separate 64 KB segments. Suitable for larger programs, providing more flexibility.
 - **Large:** Multiple separate segments for code and data, allowing large programs with extensive data structures. Supports multiple code and data segments, each up to 64 KB in size. Useful for even larger programs, but with increased complexity.

.stack

- Defines the size of the stack segment. The stack is used for storing temporary data, function parameters, return addresses, and local variables.
- 100h in hexadecimal is equal to 256 in decimal
- ; is used for comment just as // in C++

assembly

```
.stack 100h ; Allocates 256 bytes for the stack
```


.data

- Marks the beginning of the data segment, where initialized data is defined.
- This is where you can declare variables that hold values that your program will use.

```

.data
message DB 'Hello, World!', 0 ; Defines a string with a null terminator
count DW 10                  ; Defines a word-sized variable initialized to 10
```


.code

- Indicates the beginning of the code segment, where the actual instructions (the executable part of the program) are written.

```
.code  
main PROC  
    ; Your code here  
main ENDP
```


.exit

- .exit or .end
- Marks the end of the program or code segment. It can signal to the assembler or the linker that there are no more instructions or data to process.
- Marks the point in the code where the program will perform

th

exit:

mov ax, 4C00h ; DOS terminate program function

int 21h ; Interrupt to call DOS

First Assembly Language Program

```
.model small  
  
.stack 100h  
  
.data  
  
.code  
  
Mov ax, 100h  
Mov bx, 200h  
Add ax, bx  
Sub ax, 25h  
  
.exit
```


Example

```
.model large
.stack 100h      ; Define stack size
.data           ; Data segment
count DB 100    ; Define a byte variable
; Additional data segments can be defined here
.code           ; Code segment
main PROC
    ; Your code goes here
main ENDP
; Additional code segments can be defined here
.exit
```


Program to add 2 numbers

```
section .data
    num1 db 5          ; First number
    num2 db 3          ; Second number
    result db 0        ; Result variable
```

```
_start:
    ; Add the two numbers
    mov al, [num1]     ; Load num1 into AL register
    add al, [num2]     ; Add num2 to AL
    mov [result], al   ; Store the result
```


Summary

- Structure of an Assembly Language Program
- `.model small` ; Select a memory model.
- `.stack stack_size` ; Define the stack size
- `.data` ; Variable and array declarations
 - Declare variables at this level
- `.code`
- `main proc` ; Write the program main code at this level
- `main endp` ; End of program

MOV Instruction

- Transfer data from one location to another
- Between registers, between a register and memory, or between a register and an immediate (constant) value
 - `Mov AL, [num1]` or `Mov AL, 5`
- Does not perform any arithmetic operation
- It simply copies the value from the source to the destination.

MOV Instruction – Syntax

- The basic syntax of MOV instruction is: `MOV destination, source`
- **Destination:** The location where the data will be copied to (this can be a register or a memory address).
- **Source:** The location from which the data will be copied (this can also be a register, memory address, or an immediate value).

Data Movement Among Registers

- Moving Data Between General-Purpose Registers
 - `MOV AX, BX` ; Move the value in BX to AX
- Moving Immediate Values into Registers
 - `MOV CX, 10` ; Move the immediate value 10 into CX
- `Mov 10, CX` (Is that valid?)

Machine Instruction Elements

- Each instruction must have elements that contain the information required by the CPU for execution.
- These elements can be:
 - **Operation code:** Specifies the operation to be performed (e.g.. ADD, I/O). The operation is specified by a binary code, known as the operation code, or opcode.
 - ADD, SUB, MOV, or JMP translate to opcodes in machine language

Machine Instruction Elements

- **Source operand reference:** The operation may involve one or more source operands, that is, operands that are inputs for the operation.
- **Result operand reference:** The operation may produce a result. Also called destination operand.
- **Next instruction reference:** This tells the CPU where to fetch the next instruction from.

Instruction Representation

- Within the computer, each instruction is represented by a sequence of bits.
- 16 bits instruction
 - 4 bit opcode, 6 bit operand 1, 6 bit operand 2
 - 4 bit opcode, 12 bit operand
- 32 bits instruction
- 64 bits instruction

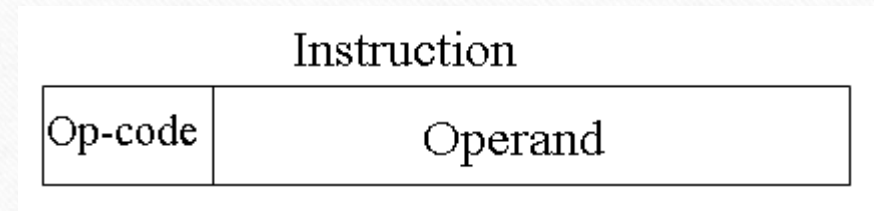
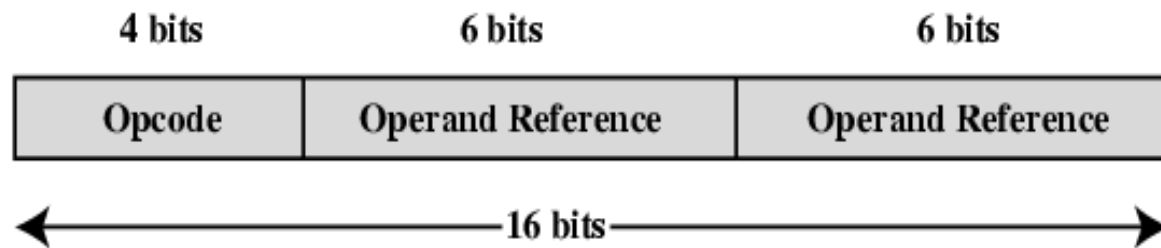


Figure: A Simple Instruction Format

Cont.

- Binary representations of machine instructions is difficult to remember.
- Use a symbolic representation of machine instructions.
- Opcodes are represented by abbreviations, called mnemonics, that indicate the operation. Common examples include:

ADD	Add
SUB	Subtract
MPY	Multiply
DIV	Divide
LOAD	Load data from memory
STOR	Store data to memory

Some Rules To Follow

- Both source and destination operands should be the same size.
- Both operands cannot be segment registers simultaneously
 - `MOV DS, SS`
 - Using two segment registers as operands in an operation could create ambiguity about which segment of memory is being referenced.
- The destination operand cannot be a CS or an IP register
 - `MOV CS, AX`
 - Changing the CS register directly could disrupt the flow of program execution
- If the destination operand is a segment register, the source operand cannot be an immediate (constant) value. Instead, copy it from some other register like AX.
 - `MOV ES, [num1]` is not valid.
- **No direct memory-to-memory moves.**
 - `mov [var1], [var2]`
 - Use register as intermediate:
 - `mov al, [var1]` and `mov [var2], al`

Addressing Modes

- Addressing modes are techniques used in assembly language to specify how the operand of an instruction is accessed.
- They determine the method of obtaining the data for the operation, whether from a constant, register, memory address, or a combination of these.

IMMEDIATE ADDRESSING MODE

- The data will be specified directly in the source operand.
- Either 16 bit hex data (4 digits) or 8 bit hex data (2 digits) can be specified in the instruction.

Example:

- MOV AX, 0004H- The 16 bit data 0004H will be moved to accumulator register.

Before Execution	After Execution
The value of AX---->A00C	The value of AX---->0004

- MOV AL, 04H- The 8 bit data 04 is moved to lower byte of accumulator register.

Before Execution			After Execution		
The value of AX---->	AH	AL	The value of AX---->	AH	AL
	A0	0C		A0	04

Note: If 16 bit data is to be specified, use AX, BX, CX, DX..

If 8 bit data is to be specified, use AL, AH, BL, BH, CH, CL, DL, DH..

DIRECT ADDRESSING MODE

- In this mode, the **16 bit offset address** or the input-output (**IO**) address (8 bit) is directly specified in the instruction.

Example:

- MOV AX, [6000H]- The data which resides in memory location 6000H and 6001H is moved to AX register.
- Assume the data in 6000H----> 04 and 6001H---->01.

Before Execution	After Execution
The value of AX---->A00C	The value of AX---->0104
Note: Every memory location can store only 8 bit data. So, to do any operation with 16 bit data, 2 memory locations are needed. The value of 1 st memory location is stored in lower byte and the value of 2 nd memory location is stored in higher byte.	

- IN AL, 80H- This instruction reads the data from the address 80H and stores it in AL register.
- Assume the data in 80H---> 05

Before Execution	After Execution
The value of AX---->A00C	The value of AX---->0105

REGISTER ADDRESSING MODE

- In this mode, **both source and destination operands** are registers.

Example:

- MOV AX, BX - The data which resides in BX register is moved/copied to AX register.

Before Execution	After Execution
The value of AX---->A00C The value of BX--->0902	The value of AX---->0902 The value of BX--->0902

- ADD AL, BL - The data in BL register and AL register are summed up and the result is stored in AL register.

Before Execution				After Execution			
AX register		BX register		AX register		BX register	
AH	AL	BH	BL	AH	AL	BH	BL
03	03	01	02	03	05	01	02

REGISTER INDIRECT ADDRESSING MODE

- The **address of the memory location** which contains the data (will be enclosed in square brackets) is specified by offset registers.
- Offset register-BX register

Example:

- MOV AX, [BX]- Assume the BX register has the value 4000H. Now, this 4000H is considered as offset address/ memory location. Then, the data in 4000H and 4001H are moved to AX register.

Before Execution			After Execution	
The value of AX---->A00C			The value of AX---->0604	
The value of BX---> 4000			The value of BX--->4000	
BX--->	4000	04		
	4001	06		

Note: If no square brackets are enclosed, it becomes register addressing mode.

INDEXED ADDRESSING MODE

- The **address of the memory location** which contains the data (will be enclosed in square brackets) is specified by index registers.
- Index registers-SI (Source Index), DI (Destination Index)

Example:

- MOV AX, [SI]- Assume the SI is pointed to the content 4000H. Then, the data in 4000H and 4001H are moved to AX register.

Before Execution	After Execution				
The value of AX---->A00C The value of SI---> 4000 SI---> <table><tr><td>4000</td><td>04</td></tr><tr><td>4001</td><td>06</td></tr></table>	4000	04	4001	06	The value of AX---->0604 The value of SI---> 4000
4000	04				
4001	06				

- MOV [DI], AX- Assume the DI is pointed to the content 6002H . Then, the data in AX register is moved to 6002H and 6003H.

Before Execution	After Execution								
The value of AX---->A00C DI---> <table><tr><td>6002</td><td>09</td></tr><tr><td>6003</td><td>0E</td></tr></table>	6002	09	6003	0E	The value of AX---->A00C DI---> <table><tr><td>6002</td><td>0C</td></tr><tr><td>6003</td><td>A0</td></tr></table>	6002	0C	6003	A0
6002	09								
6003	0E								
6002	0C								
6003	A0								

Physical Address to Logical Address

- **Logical Address:** An address generated by the CPU during program execution, consisting of a **segment** and an **offset**.
- **Physical Address:** The actual location in physical memory (RAM) where data is stored.
- Physical address will be calculated by Segment address and Offset
- Both Segment address and Offset size is 16 bits.
- So Physical address is calculated by **Segment * 10h + offset address**
- 10h = 16

Example

- Segment: 0x1234
- Offset: 0x5678
- Step 1: Convert the Segment to a Physical Address
 - Base Physical Address=Segment×16
 - 0x1234 = 4660 (decimal)
 - 4660×16=74560 = 0x12340
- Base Physical Address=0x12340

Example

- Step 2: Add the Offset
- Physical Address=Base Physical Address + Offset
 - $0x5678 = 22136$
- Physical Address= $74560 + 22136 = 96696$
 - $96696 = 0x179B8$
- Physical Address= $0x179B8$

Practice Questions

- 0x2000:0x0050 segment:offset
- 0x3000:0x00C0
- 0x1234:0x00A0
- 0x4000:0x0100
- 0x1A2B:0x0005